

DLQ 운영 절차

재큐잉 판단 기준과 드랍 정책

DLQ 앞에 선 운영자가 무엇을 어떻게 결정하는가
강의 25min + 실습 30min

DLQAdminService

requeue 판단

drop 정책

IncidentService

DLQ 알림 수신 — 운영자의 첫 질문

THREE ACTIONS WHEN DLQ ALERT ARRIVES — NOT "JUST REQUEUE"

DLQ 메시지 수신



listPending() + 로그 조회
_reason · failedAt · txHash 확인



일시적 외부 장애

VASP 다운
DB 일시 불가

→ 복구 후 **requeue**

영구 실패

코드 버그
스키마 불일치

→ 수정 배포 후 **requeue**

드랍 대상

이미 처리됨
파싱 불가

→ 감사 로그 후 **drop**

❌ 절대 안 되는 것

"일단 **requeue**"

→ 원인 모른 채 재투입
→ 또 실패 → 또 DLQ
→ 무한 루프

✅ 반드시 지킬 것

원인 파악 → 조치 → **requeue**
이 순서를 절대 바꾸지 않는다

핵심 — DLQ는 "일단 **requeue**"가 아니다. **requeue**는 원인이 해소됐다는 확신이 있을 때만 실행한다.

재큐잉(requeue) 판단 기준

WHEN TO REQUEUE — THREE CONDITIONS, ALL MUST BE MET

조건	판단 근거 (_reason 확인)	조치 순서
일시적 외부 장애	timeout / connection refused	외부 서비스 복구 확인 → requeue
코드 버그 수정됨	배포 이력 확인 + 테스트 통과 확인	배포 완료 후 requeue
데이터 정정 완료	관련 시스템에서 정정 완료 확인	정정 확인 후 requeue

requeue 전 3가지 확인 — 셋 다 YES여야 requeue 가능

- ① processed_events 에 같은 txHash가 없다
있으면 → 이미 처리됨 → drop 대상
- ② 실패 원인이 해소됐다
장애 복구 / 코드 배포 / 데이터 정정
- ③ requeue하면 성공할 근거가 있다
추측이 아닌 확인된 사실

하나라도 NO → requeue 금지. 원인 해소 후 재판단하거나 drop을 검토한다.

드랍(drop) 판단 기준

WHEN TO DROP — FOUR CONDITIONS, ANY ONE IS SUFFICIENT

조건	판단 근거	주의사항
이미 원장에 반영됨	<code>processed_events</code> 에 동일 txHash+logIndex 존재	역등성 처리 완료 — drop 안전
파싱 불가 메시지	<code>_reason</code> 에 JSON parse error / unknown field	발신 시스템에 포맷 오류 통보 필수
TTL 초과	이벤트 발생 시각 기준 N시간 초과	금융 이벤트는 TTL 길게 설정 권장
테스트/장애주입 데이터	<code>eventType</code> 에 test_prefix 등 식별	운영 환경 유입 방지 프로세스 점검

⚠ 드랍은 되돌릴 수 없다 — drop 전 반드시 3가지

- 1 감사 로그에 드랍 이유 기록
DLQAdminService.drop()이 자동 처리
- 2 원본 메시지 전체 내용 백업
S3 또는 별도 DB에 원본 페이로드 보존
- 3 2인 확인 원칙
금융 시스템 내부 통제 요구사항

알림 단계 설계 — 언제 누구에게 무엇을

TIERED ALERT DESIGN — SEVERITY BY MESSAGE VOLUME AND RATE

1건

DLQ 적재



담당자 Slack DM + 채널 알림

메시지 ID · 이벤트 타입 · 실패 원인 · 현재 DLQ 수

5건+

10분 내 누적



팀 채널 @채널 태그

단발성이 아닌 반복 장애 가능성 — 팀 전체 인지 필요

20건+

누적



서비스 관리자 즉시 호출 (PagerDuty · 전화)

대량 장애 · 시스템 전반 영향 가능성 — 에스컬레이션 필수

알림 필수 항목 — DLQ ID · 이벤트 타입 · 원인 · 발생 시각 · 현재 DLQ 누적 수. 건수 없으면 심각도 판단 불가.

DLQAdminService — 운영 API 구조

ADMIN SERVICE — WRAPS DLQHANDLER WITH VALIDATION + AUDIT LOG

listPending() — 이벤트 타입 필터 지원

```
async listPending(options?: {
  eventType?: string; limit?: number
}) {
  const all = await this.dlqHandler.listPending(
    options?.limit ?? 100
  );
  const filtered = options?.eventType
    ? all.filter(i =>
        i.event['eventType'] === options.eventType
      ) : all;

  return filtered.map(item => ({
    dlqId: item.messageId,
    eventType: item.event['eventType'] ?? 'unknown',
    reason: item.reason,
    failedAt: item.failedAt,
    txHash: item.event['txHash'] ?? null,
  }));
}
```

DLQAdminService 의존성 3가지

DLQHandler

move / listPending / requeueMessage / drop

Database

processed_events 조회 — 원장 중복 확인

AuditLogService

모든 운영 행위 기록 (operator, action, timestamp)

requeue API — 원장 확인 + 감사 로그 필수

REQUEUE WITH LEDGER GUARD AND AUDIT TRAIL

```
async requeue(
  dlqId: string, operator: string, reason: string,
): Promise<{ newMessageId: string }> {

  // 1. DLQ에서 메시지 확인
  const items = await this.dlqHandler.listPending();
  const target = items.find(i => i.messageId === dlqId);
  if (!target) throw new Error(`DLQ 메시지 없음: ${dlqId}`);

  // 2. 이미 원장에 반영됐는지 확인 (drop해야 할 케이스 방어)
  const txHash = target.event['txHash'];
  if (txHash) {
    const alreadyProcessed = await this.db.query(
      `SELECT 1 FROM processed_events WHERE tx_hash = $1 LIMIT 1`, [txHash],
    );
    if (alreadyProcessed.rows.length > 0)
      throw new Error('이미 원장에 반영됨. requeue 대신 drop을 사용하세요.');
```

```
}
```

drop API — 감사 로그 + 원본 백업 필수

DROP WITH MANDATORY AUDIT LOG — IRREVERSIBLE OPERATION

```
async drop(
  dlqId: string, operator: string, reason: string,
): Promise<void> {
  const items = await this.dlqHandler.listPending();
  const target = items.find(i => i.messageId === dlqId);
  if (!target) throw new Error(`DLQ 메시지 없음: ${dlqId}`);

  // 드랍 전: 전체 페이로드 감사 로그에 백업
  await this.auditLog.append(operator, 'DLQ_DROP', dlqId, {
    reason,
    originalEvent: target.event, // 전체 페이로드 보존
    originalReason: target.reason,
    failedAt: target.failedAt,
  });

  // 실제 삭제
  await this.dlqHandler.drop(dlqId);
}

// DLQHandler.drop() ($11에 추가)
async drop(dlqMessageId: string): Promise<void> {
  const deleted = await this.redis.xdel(
    this.dlqStreamKey, dlqMessageId
  );
  if (deleted === 0)
    throw new DLQMessageNotFoundError(dlqMessageId);
}
```

drop 순서 — 반드시 이 순서

- 1 감사 로그에 원본 전체 백업
↓
- 2 xdel로 DLQ에서 영구 삭제

감사 로그로 추적 가능

```
auditLog.query({
  action: 'DLQ_DROP',
  resourceId: dlqId
});
// afterState에 전체 원본 페이로드
```


운영 체크리스트 — DLQ 알림 수신 시

OPERATOR CHECKLIST — STEP BY STEP FROM ALERT TO RESOLUTION

1 DLQ 조회

`GET /admin/dlq/pending`

건수 · 이벤트 타입 · 원인 파악

2 _reason 분석

timeout/conn → 외부 장애

parse error → 포맷 오류

business rule → 코드 버그

3 txHash 있으면

`processed_events` 조회

이미 처리됐으면 → drop

4 원인 미해소 → **requeue** 금지

원인 해소 확인 후에만 requeue

5 drop 시 감사 로그 이유 기록

2인 확인 권장 (금융 내부 통제)

6 requeue 후 5분 내 확인

XACK → 정상 ✓

또 DLQ → 즉시 에스컬레이션

2회 DLQ 진입 = 비즈니스 로직 버그 의심 — 같은 메시지가 requeue 후 또 DLQ로 가면 원인 파악이 불충분. 즉시 에스컬레이션.

IncidentService — 워크시트의 디지털 버전

INCIDENT SERVICE — AUTOMATE WHAT CAN BE AUTOMATED, HUMAN FOR THE REST

아날로그 워크시트 → 디지털 자동화

운영자 직접 입력 (판단 필요)

- 원인 분류
- requeue / drop 결정
- drop 이유
- 2차 확인자

✅ 자동 수집 (API 조회)

- queueStats 스냅샷
- dlqItems 목록
- processed_events 중복 여부
- 인시던트 감지 시각

create(type)

현재 상태 자동 스냅샷
queueStats + dlqItems
alreadyInLedger 자동 확인
알림 발송



addAction(id, action)

운영 행위 기록
REQUEUE / DROP
operator + note 추적
status → INVESTIGATING



resolve(id, note)

인시던트 종료
resolvedAt 기록
종료 알림 발송
status → RESOLVED

IncidentService 사용 흐름

DIGITAL WORKFLOW — API CALLS REPLACING PAPER WORKSHEET

워크시트
1단계

초기 상태 확인

```
POST /admin/incidents { type: 'DLQ', detectedBy: 'alert-bot' }  
// 응답: queueStats + dlqItems(alreadyInLedger 포함) 자동 채워짐
```

워크시트
3-4단계

원인 파악 + 조치

```
POST /admin/incidents/:id/actions  
{ operator: 'kim@kyobo.com', action: 'DROP',  
  targetId: '1714320-0',  
  note: 'alreadyInLedger=true 확인. 원장 정상, 중복 이벤트 드랍' }
```

워크시트
5-6단계

사후 확인 + 기록

```
POST /admin/incidents/:id/resolve  
{ operator: 'kim@kyobo.com',  
  resolutionNote: '중복 이벤트 드랍 완료. 발신 시스템 중복 발행 원인 조사 중' }
```

자동화의 목표 — 운영자가 판단해야 하는 부분(원인 분류, requeue/drop 결정)만 남기고, 나머지는 시스템이 채운다.

DLQ 메시지 분석 워크시트

PAPER WORKSHEET — FILL IN ORDER = DIAGNOSIS IN ORDER

기본 정보

DLQ ID _____
이벤트 타입 _____
현재 DLQ 수 _____

원인 분류

- ☐ 일시적 외부 장애 (서비스명: _____)
- ☐ 코드 버그 (위치 추정: _____)
- ☐ 스키마/파싱 오류 (필드: _____)
- ☐ 비즈니스 규칙 위반

원장 중복 확인

txHash: _____
☐ 이미 반영됨 → drop 대상
☐ 미반영 → 재처리 대상

조치 결정

- ☐ requeue
조치 완료: _____
- ☐ drop
drop 이유: _____
2차 확인자: _____

실행 기록

- ☐ POST /admin/dlq/requeue → 새 ID: ____
- ☐ DELETE /admin/dlq/drop

사후 확인 (5분 후)

- ☐ XACK 정상 완료 ✓
- ☐ 또 DLQ 진입 → 에스컬레이션 ⚠

주간 DLQ 리뷰 — 패턴에서 개선 과제 찾기

WEEKLY REVIEW — REPEAT PATTERNS ARE CODE/INFRA IMPROVEMENT SIGNALS

집계 항목

총 DLQ 적재 _____건
requeue 건수 _____건
drop 건수 _____건
2회 이상 DLQ 진입 _____건

원인 분포

외부 장애(timeout): _____건 (____%)
코드 버그: _____건 (____%)
파싱 오류: _____건 (____%)
먹등성(이미 반영): _____건 (____%)

⚠ 반복 패턴 감지 기준

동일 원인 3회 이상 = 코드/인프라 개선 필요 신호

2회 이상 DLQ 재진입 = 원인 분석 불충분

외부 장애 비율 높음 = SLA 재협상 필요

WeeklyReportService 자동 집계

매주 월요일 09:00 KST 자동 발송

유형별 건수 · 평균 해소 시간

반복 패턴(동일 원인 3회+) 자동 감지

리뷰 목적 — 주간 리뷰는 단발 장애 대응이 아니라 **패턴 발견**이다. 반복 패턴은 코드·인프라·프로세스 개선 신호.

실습 — CLI로 조회 · 재큐잉 · 드랍

HANDS-ON — CURL COMMANDS FOR ALL THREE OPERATIONS

① DLQ 조회

```
# 전체 조회
curl http://localhost:3000\
/admin/dlq/pending

# 이벤트 타입 필터
curl http://localhost:3000\
/admin/dlq/pending\
?eventType=NFT_MINTED
```

```
[{
  "dlqId": "1714320-0",
  "eventType": "NFT_MINTED",
  "reason": "VASP timeout",
  "txHash": "0xabc..."
}]
```

② requeue

```
curl -X POST \
/admin/dlq/requeue \
-H "Content-Type: application/json" \
-d '{
  "dlqId": "1714320-0",
  "operator": "kim@kyobo",
  "reason": "VASP 복구 확인"
}'
```

```
{
  "newMessageId": "1714400-0"
}
```

③ drop

```
curl -X DELETE \
/admin/dlq/drop \
-H "Content-Type: application/json" \
-d '{
  "dlqId": "1714320-0",
  "operator": "kim@kyobo",
  "reason": "processed_events 확인"
}'
```

```
{ } // 204 No Content
// 감사 로그 자동 기록
// 원본 페이로드 보존
```

완료 기준 — requeue 후 DLQ 건수 1 감소 · 이미 반영된 txHash → requeue 에러 응답 · drop 후 감사 로그에 원본 페이로드 기록 확인

S51 완료 기준

ACCEPTANCE CRITERIA — ALL MUST PASS BEFORE SIGN-OFF

☐ GET /admin/dlq/pending — 전체 목록 및 이벤트 타입 필터 동작

☐ POST /admin/dlq/requeue — 재큐잉 후 DLQ 건수 1 감소 확인

☐ POST /admin/dlq/requeue — 이미 원장에 반영된 txHash → **에러 응답** 확인

☐ DELETE /admin/dlq/drop — DLQ 건수 1 감소 + **감사 로그에 원본 페이로드 기록** 확인

☐ requeue · drop 모두 감사 로그에 **operator 필드 포함** 확인

S51의 핵심 — DLQ는 "일단 requeue"가 아니다. **원인 파악 → 조치 → requeue** 순서.
drop은 되돌릴 수 없다. 감사 로그 + 2인 확인이 필수. 모든 운영 행위는 추적 가능해야 한다.

S51 핵심 정리

KEY TAKEAWAYS — DLQ OPERATIONS, JUDGMENT, AND GOVERNANCE

운영 3원칙

1. 원인 파악 먼저 — "일단 requeue"는 무한 루프의 시작
2. 원장 확인 필수 — txHash가 processed_events에 있으면 drop
3. 모든 행위 추적 — drop은 감사 로그 + 2인 확인

requeue 조건 — 셋 다 YES

- ① 원장 미반영 ② 원인 해소 ③ 성공 근거 있음

drop 조건 — 하나라도 해당

- ① 원장 반영됨 ② 파싱 불가 ③ TTL 초과 ④ 테스트 데이터

알림 → IncidentService 자동화

1건 알림 → 5건 채널 → 20건 에스컬레이션
워크시트 자동 수집 + 운영자 판단만 입력

S11의 DLQHandler는 "어떻게 격리하는가", S51은 "운영자가 격리된 메시지 앞에서 무엇을 결정하는가"다.
기술이 아니라 판단 기준과 거버넌스가 핵심.